

TP2 - Sistemas Operativos

1. En el diagrama de estados de un proceso, ¿dónde interviene el S.O.? ¿Cómo debería ser el diagrama para ver todos los procesos y el S.O.?

Antes que nada, debemos ver el diagrama de estados que tiene un proceso presentado por la cátedra, con una breve descripción de los mismos.



- Nuevo: el proceso es creado.
- Corriendo (ejecutando): las instrucciones están siendo ejecutadas.
- Espera: el proceso está esperando que ocurra algún evento.
- Listo: el proceso está esperando ser asignado a la CPU.
- Terminado: el proceso ha finalizado su ejecución.

Entonces en base a estas descripciones podemos analizar los diferentes momentos donde interviene el SO. Cuando un proceso está corriendo y tiene que esperar a una E/S o evento para continuar, el SO debe darse cuenta que el proceso no puede continuar y lo pausa, o a veces en algunos sistemas el proceso mismo se pone en espera. También cuando un proceso está listo para correr y que lo asignen a un procesador, entrando en juego aquí la planificación de procesos que implemente ese sistema operativo. Otro de ellos es cuando se pasa de estado corriendo a listo, en este caso el SO (mediante el programador de CPU) determina que el proceso ya tuvo su tiempo en el CPU y es el turno de otros procesos, o que hubo una interrupción, en cuyo caso realiza el cambio de contexto.

El diagrama para ver todos los procesos y el SO adecuado tendría que ser el mismo que está en el gráfico de arriba. Si bien hay otro más simple, esos estados Susp-listo y Susp-esp incluyen a su vez la implementación de una memoria virtual, ya que hasta ahora, en el análisis se ha supuesto que la memoria virtual no está en uso y que un proceso está completamente en la memoria principal o completamente fuera de ella.

- Susp-listo: El proceso está en memoria secundaria pero está listo para ser ejecutado cuando sea cargado a la memoria principal.

- Susp-esp: el proceso está en memoria secundaria y a la espera de un evento.

2. ¿Cuáles son, normalmente, los sucesos que llevan a la creación de un proceso?

- Inicialización del sistema.
- Ejecución de una llamada al sistema de creación de procesos mediante un proceso que se está ejecutando.
- Una solicitud de usuario para crear un nuevo proceso.
- Iniciación de un trabajo por lotes.

Los eventos de arriba son los que otorga la bibliografía de Tanenbaum. Stallings muestra estos 4 con su respectiva explicación:

New batch job	The OS is provided with a batch job control stream, usually on tape or disk. When the OS is prepared to take on new work, it will read the next sequence of job control commands.
Interactive log-on	A user at a terminal logs on to the system.
Created by OS to provide a service	The OS can create a process to perform a function on behalf of a user program, without the user having to wait (e.g., a process to control printing).
Spawned by existing process	For purposes of modularity or to exploit parallelism, a user program can dictate the creation of a number of processes.

3. ¿Cuáles son los pasos que lleva a cabo un sistema operativo para crear un nuevo proceso?

En el proceso de creación de un nuevo proceso, primero se asigna un identificador único y se añade una entrada a la tabla de procesos principal (contiene una entrada por proceso). Luego, se asigna espacio para el proceso, incluyendo su imagen de proceso, el espacio de direcciones privado para programas y datos, y la pila del usuario, lo cual puede definirse por defecto o basarse en una solicitud del usuario. Si el proceso es creado por otro proceso, el proceso padre puede proporcionar los valores necesarios para la creación. Después, se inicializa el bloque de control del proceso, que contiene el identificador del proceso, el estado inicial del procesador, como el contador de programa y los punteros de la pila del sistema, y la información de control del proceso con valores por defecto o solicitudes específicas. Posteriormente, se crean los enlaces apropiados, como la inclusión del proceso en la cola de planificación adecuada (algunas de ellas son cola de procesos listos, de procesos bloqueados o en espera, de procesos suspendidos, de planificación a largo plazo, a corto plazo o de planificación a mediano plazo). Finalmente, se expanden otras estructuras de datos si es necesario, como archivos contables para evaluar el rendimiento o facturar.

5. Describe las diferencias entre planificadores de corto, mediano y largo término.

Antes que nada, me gustaría especificar brevemente a cada uno de los planificadores. Por un lado tenemos al planificador corto, o también conocido como planificador de CPU el cual selecciona qué procesos deberían ser próximamente ejecutados y colocados en la CPU. Respecto a los planificadores de largo término o también conocidos como planificador de jobs, selecciona qué procesos deberían ser puestos en la cola de listos. Finalmente, el planificador de mediano término es quien se encarga de hacer el swapping. Este swapping

es con el objetivo de ahorrar espacio en la memoria principal, dando lugar a otros procesos con mayor prioridad, reduciendo así el grado de la multiprogramación, donde se manda un proceso un nivel inferior de MP (Disco), reemplazándolo por otro. Cabe recalcar que el estado de este proceso que se manda al disco se guarda como lo pudimos ver en los estados agregados arriba.

La principal diferencia por un lado es que el de largo término puede llegar a ser muy lento, ya que es invocado poco frecuentemente (segundos, minutos), mientras que el planificador de corto término tiene que ser lo más rápido posible (nanosegundos, microsegundos), ya que es invocado muy frecuentemente. Otra diferencia podría ser que el planificador de largo término controla el grado de multiprogramación nombrado antes (el cual es el número de procesos actualmente en la memoria), mientras que el otro no.

6. Suponga que cuatro procesos se están ejecutando en un sistema con un sólo procesador, las direcciones de inicio de cada uno son las siguientes: 4050, 3200, 5000 y 6700. Las trazas individuales se muestran en el cuadro 1. Realice la traza de entrelazado de ejecución de los procesos. Asuma que el dispatcher es invocado cada 5 instrucciones o por una interrupción y el dispatcher ejecuta 4 instrucciones.

Antes de comenzar con la traza, me gustaría explicar qué es lo que hace el dispatcher el cual es un planificador de corto plazo. El dispatcher es el módulo que otorga el control del núcleo de la CPU al proceso seleccionado por el planificador de la CPU. Esta función implica lo siguiente:

- Cambiar el contexto de un proceso a otro
- Cambiar al modo de usuario
- Saltar a la ubicación adecuada en el programa de usuario para reanudar ese programa

El despachador debe ser lo más rápido posible, ya que se lo invoca durante cada cambio de contexto. El tiempo que tarda el despachador en detener un proceso y comenzar a ejecutar otro se conoce como latencia de despacho. Se puede ver que tan a menudo se realizan los cambios con el comando en Linux "vmstat".

<timeout> → 4 instrucciones

Traza: 4050 - 4051 - 4052 - 4053 - 4054 - <timeout> - 100 - 101 - 102 - 103 - 3200 - 3201 - 3202 - 3203 - 3204 - <timeout> 100 - 101 - 102 - 103 - 5000 - 5001 - 5002 - 5003 - 5004 - <timeout> - 6700 - 6701 - 6702 - <I/O> - 100 - 101 - 102 - 103 - 4055 - 4056 - 4057 - 4058 - 4059 - <timeout> - 100 - 101 - 102 - 103 - 3205 - 3206 - <I/O> - 100 - 101 - 102 - 103 - 5005 - 5006 - 5007 - 5008 - 5009 - <timeout> - 100 - 101 - 102 - 103 - 4060 - <interrupción, finaliza proceso> - 100 - 101 - 102 - 103 - 5010 - <interrupción, finaliza proceso>

7. La mayor parte de la problemática de la administración del procesador puede describirse con ayuda de un diagrama de transición de estados de los procesos, como el que se expone en la figura 1. El objetivo de este ejercicio es analizar en detalle las relaciones existentes entre los procesos, las rutinas de atención de interrupciones y el planificador de procesos. Debe describir cómo interactúan las rutinas de atención de interrupciones, el planificador de procesos y la base de datos necesaria para cumplir con el diagrama de transiciones. O sea, debe indicar:

- ¿Cuáles son los eventos que provocan las transiciones 1, 2, 3 y 4.?



Figura 1: Diagrama de Estado de Procesos

Inicialmente, cuando se crea un proceso y se encuentra en estado “nuevo”, el planificador de largo término determina si desde la cola de jobs envía a la cola de listos el proceso, y posición a los procesos en memoria para que estén en estado “listo”, esperando a ser asignado por la CPU.

Los eventos que provocan la **transición 1** es según Stalling nos dice que lo más común es porque un proceso corriendo alcanzó el tiempo máximo de ejecución sin ser interrumpido. Prácticamente, todos los sistemas operativos multiprogramación imponen esta disciplina del tipo de tiempo. Sin embargo, hay otras causas de este tipo de transición, que no están implementadas en todos los sistemas operativos. Una de ellas es el caso donde un SO asigna diferentes niveles de prioridad a los procesos (como Android). Por ejemplo, supongamos que un proceso A está corriendo con un nivel de prioridad determinado, y hay otro proceso B con un nivel de prioridad mayor al de A pero este está bloqueado (en espera). Si el SO indica que el proceso B está “listo”, entonces se puede interrumpir al proceso A y despachar el proceso B (se dice que el SO “preempted” al proceso A). Por último, un proceso puede ceder voluntariamente el control del procesador. Un ejemplo es un proceso en segundo plano que realiza alguna función de contabilidad o mantenimiento periódicamente. El nombre de los eventos pueden ser una interrupción o un timeout.

Los que provocan la **transición 2** es cuando es tiempo de seleccionar un proceso para ponerlo en ejecución, donde el SO elige uno de los procesos en estado “listo” de la cola de listos en MP. Este es el trabajo del planificador o el despachador.

Estas dos son causadas por el planificador de procesos, una parte del sistema operativo que, sin que el proceso lo sepa, hace la transición. Tanenbaum dice más de lo mismo, que la transición 1 ocurre cuando el planificador decide que el proceso en ejecución ha estado corriendo suficiente tiempo, y es momento de que otro proceso tenga tiempo de CPU. La transición 2 ocurre cuando todos los otros procesos han tenido su turno y es tiempo de que el primer proceso obtenga nuevamente la CPU. El tema de la planificación, es decir, decidir qué proceso debe ejecutarse, cuándo y durante cuánto tiempo, es importante. Se han ideado muchos algoritmos para tratar de equilibrar las demandas competidoras de eficiencia para el sistema en su conjunto y de equidad para los procesos individuales.

Respecto a la **transición 3** se da cuando el proceso solicita algo por lo que debe esperar. Una solicitud al SO es usualmente de la forma con los servicios, system call. Por ejemplo, un proceso pide un servicio desde el SO y el SO no está preparado para realizarlo de inmediato. Puede pedir un recurso, tales como una archivo o una sección compartida de la memoria virtual, que no esté en ese momento disponible. A su vez, el proceso puede requerir de un evento o de una entrada E/S que se tiene que resolver antes de poder continuar. En la comunicación entre dos procesos, un proceso tendría que estar bloqueado cuando está esperando que el otro le provea data o le mande un mensaje.

Otras bibliografías nos dicen que en algunos sistemas, el proceso puede ejecutar una llamada al sistema, como pausar (pause), para entrar en un estado bloqueado. En otros sistemas, incluidos UNIX, cuando un proceso lee de una tubería o archivo especial (por ejemplo, un terminal) y no hay entrada disponible, el proceso se bloquea automáticamente.

Finalmente, la **transición 4** ocurre cuando un proceso está en “espera” y lo mueven a “listo” ya que el evento por el que estaba esperando ocurre y satisface el requerimiento. Puede darse cuando la señal de I/O que el proceso esperaba se dio. Entonces, el proceso sale de la cola de I/O, y entra en la cola de listo, pasando además a ese estado, y se queda esperando hasta que se le asigne un procesador.

8. Describa las acciones que realiza el kernel cuando cambia el contexto entre procesos.

Cuando ocurre una interrupción, como vimos arriba los procesos cambian de estado, es decir cambia el contexto. Para ello lo que hace el sistemas es guardar el contexto actual del proceso que se está ejecutando en la CPU, para poder restaurar dicho contexto cuando su procesamiento concluya, suspendiendo el proceso y reanudándose después. Donde lo guarda es en el PCB del proceso.

Cuando ocurre un cambio de contexto, el kernel guarda el contexto del proceso antiguo en su PCB y carga el contexto almacenado del nuevo proceso que se ha decidido ejecutar. El tiempo dedicado al cambio de contexto es tiempo desperdiciado y depende del soporte del hardware. Si hay más procesos activos que conjuntos de registros, el sistema recurrirá a copiar los datos de los registros en y desde la memoria. También, cuando más complejo es el sistema operativo, más trabajo debe realizar durante un cambio de contexto.

9. ¿Un proceso puede esperar simultáneamente a más de un suceso o evento? Justifique su respuesta.

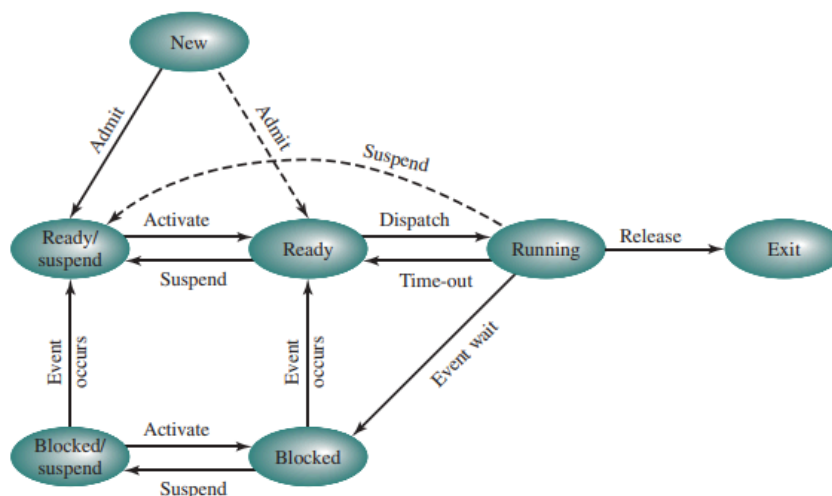
Si que puede, ya que si estamos en sistema multi hilado (como la mayoría de los de la actualidad), tendremos varios hilos donde cada uno puede estar en un estado distinto, y justamente ambos pueden estar en el estado de esperando.

En muchos sistemas, un proceso puede estar esperando múltiples señales o eventos, pero al final se desbloqueará cuando una de esas señales llegue, no todas simultáneamente. El proceso no se desbloquea de manera simultánea por varios eventos, sino que el sistema operativo administra las señales, y cuando una llega, el proceso es despertado.

10. Considere el diagrama de estados, que está formado por siete estados (revise el capítulo 3 del libro de Stallings). Suponga que el dispatcher del sistema operativo tiene que elegir un proceso. Hay procesos en el estado Listo y Listo/Suspendido, y que al menos un proceso en el estado Listo/Suspendido tiene mayor prioridad que cualquiera de los procesos en el estado Listo. Dos políticas diferentes se pueden considerar:

- 1- siempre se elige un proceso del estado Listo, para reducir al mínimo el intercambio, y
- 2- siempre se dará preferencia a los procesos de mayor prioridad.

Sugiera una política intermedia que trate de equilibrar la prioridad y el rendimiento.



Una política intermedia que consideraría, es que si la memoria secundaria en la que se encuentra el proceso el cual su prioridad es mayor está en una memoria tipo SSD, entonces minimizaría el swapping ya que las memorias de este tipo son bastantes decentes en cuanto a velocidad. En cambio, si el disco no es sólido (SSD) y es duro (HDD), entonces sí creo que lo más conveniente es traer los procesos que se encuentren aquí listos y con gran prioridad a memoria principal. Otra sería por ejemplo usar n procesos seguidos "listos" con menor prioridad, y cada cierto tiempo realizar un swap y traer otros más importantes. También se puede jugar con las prioridades, si estas no son significativas no se realiza el swap, en cambio si la diferencia es notoria es más conveniente traerlas a memoria principal.

Nota: Muchos de los procesos listos, a veces requieren más memoria principal, y es por eso que se mandan a otros a la memoria secundaria.

11. ¿Cómo se pueden comunicar los procesos? Brinde un ejemplo en el cual considere necesario que 2 procesos se comuniquen.

Para que los procesos se comuniquen antes que nada tienen que ser procesos cooperativos, es decir se pueden ver afectados o afectar a los demás procesos que se ejecutan en el sistema. Para la cooperación entre procesos se requieren justamente de mecanismos de comunicación, existiendo dos modelos fundamentales: **memoria compartida** y **pasaje de mensajes**.

Un ejemplo en que dos procesos se comuniquen puede ser mediante una gran tarea que se necesitaba hacer, y se la subdividió en subtareas para aprovechar los distintos núcleos del procesador. Esto implica que estas subtareas necesitan comunicarse para pasarse información que necesitan en común, para coordinarse o sincronizarse. Otro ejemplo es en el famoso problema productor, consumidor. A su vez, podría pasar que dos procesos necesitan la misma pieza de información.

12. Considere la comunicación sincrónica y asincrónica entre procesos. Presente ventajas y desventajas de cada una.

La comunicación sincrónica es aquella que se considera que el pasaje de mensajes es bloqueante, donde el proceso que se envía se bloquea hasta que el proceso receptor o el mailbox recibe el mensaje y a su vez el receptor se bloquea hasta que hay un mensaje disponible.

Por otro lado, la comunicación asincrónica es la recíproca en la que el pasaje de mensajes es no bloqueante (send y receive no bloqueante). Aquí los envíos son tal que el proceso transmisor envía mensajes y continúa operando y el receptor extrae un mensaje válido o un mensaje nulo.

Entonces sabiendo esto, se puede analizar que las ventajas del sincrónico por un lado es que siempre nos aseguramos de tener controlado los send y receive y podemos evitar la saturación de alguno de los dos. Claramente se evita muchos problemas parando el proceso esperando a que pase lo que nosotros queremos, sin embargo, esta espera puede ser contraproducente y el tiempo en algunos casos podría ser bastante alto ya que el proceso no puede hacer otra cosa mientras. Es por eso que la ventaja del asincrónico se ve afectada en los tiempos de ejecución donde una vez que envía el mensaje puede seguir funcionando. La desventaja de esto es que hay que hacer mecanismos adicionales para asegurar que se coordinen los mensajes, y que se envíen y reciban correctamente, lo cual es más costoso y difícil de implementar por el aumento de la complejidad. Al final, según la profe lo mejor es basarse en las prioridades.

13. Considere un esquema de comunicación entre procesos basado en mailboxes.

a) Suponga un proceso P que desea esperar por dos mensajes, uno desde el mailbox A y el otro desde el mailbox B. ¿Cuál secuencia de send y receive debería ejecutar?

b)

14. ¿Cuáles son las características más importantes de los threads? ¿En qué casos los utilizaría? ¿Cuál es la diferencia de los threads a nivel kernel y los que son implementados a nivel usuario?

Las características más importantes de los threads (unidad básica de utilización de la CPU) es que en primer lugar si tenemos varios hilos para un mismo proceso, entonces estos comparten los mismos recursos tales como la sección de código, la sección de datos y otros recursos como archivos y señales (aún así cada uno tiene registros, ID, PC y pila propios). Es decir que te permite realizar actividades diferentes en el mismo espacio de direcciones.

Compartir memoria de esta forma a su vez abarata los costos, ya que la asignación de memoria y recursos para la creación de procesos es costosa. Por lo que al tener esta compartición de recursos, es más económico realizar y crear cambios de contextos entre los hilos.

Otra de las características importantes es la de la capacidad de respuesta, donde el uso de múltiples hilos permite que un programa continúe ejecutándose incluso aunque parte de él esté bloqueado o realizando una operación muy larga. Y en arquitecturas multiprocesador los hilos pueden ejecutarse en paralelos en diferentes procesadores, pudiendo subdividir las tareas para aquellas tareas muy grandes (Ejemplo: uno hace E/S mientras que el otro está procesando).

La diferencia entre los threads a nivel kernel y a nivel usuario más básica es que para los hilos a nivel usuario, el sistema operativo NO sabe que existen, ya que estos son administrados por librerías. Por otro lado, los de nivel kernel sí saben en todo momento el sistema operativo que existen ya que justamente es el kernel quien los administra. Otro de ellos es que si estamos en un caso donde supongamos que recibimos una E/S, **un hilo a nivel usuario en este caso haría esperar al proceso, mientras que en el otro un hilo puede esperar y otros seguir en ejecución porque los hilos a nivel kernel pueden ejecutar en paralelo.**

15. ¿Cuáles recursos son utilizados cuando un thread es creado? ¿Y en el caso de un proceso?

Cuando un thread es creado los recursos que son utilizados son todos los que tiene el proceso tales como la porción del código, los archivos y señales y datos. Y por cada hilo propio, su ID, el conjunto de registros y la pila.

En cuanto a los procesos, los recursos utilizados dependen de la política que utilice el SO, pero en UNIX lo que se implementa es que a cada proceso hijo se le asigna todos los recursos del padre y su mismo espacio direccionable (Stack, Heap, Código, etc) e independientemente un PID nuevo y PCB que es la representación del proceso en el sistema.

16. Proporcione dos ejemplos de programación en los que los mecanismos de multithread no proporcionen un mejor rendimiento que una solución monothread.

Primer ejemplo: todos aquellos programas los cuales no son paralelizables, y si o si necesitan de un procedimiento secuencial.

Segundo ejemplo: como vimos en la actividad de brainstorming, a veces cuando estamos resolviendo alguna problemática y es un caso de prueba pequeño, a veces se puede considerar utilizar un solo hilo y no tendría sentido gastar más recursos cuando se puede resolver de manera simple.

17. Una solución multithreaded utilizando threads a nivel de usuario. ¿presenta mejor rendimiento (performance) en un sistema multiprocesador que en un sistema monoprocesador?

Claramente que no presenta mejor rendimiento, ya que al tener todos los hilos a nivel de usuario, da igual si tengo 20 en total, que estos van a estar ligados a un sólo proceso, es decir, a una CPU. Entonces como no se pueden ver los hilos a nivel kernel claros que son los que administra el SO, entonces no se puede paralelizar y asignarlos todos al mismo kernel.

Explicado de otra manera, por lo tanto, aunque estés ejecutando múltiples threads dentro de un proceso en un sistema multiprocesador, todos esos threads siguen siendo parte de un único proceso desde la perspectiva del sistema operativo. Debido a que el kernel del sistema operativo no puede "ver" los hilos de usuario, todos los threads de ese proceso serán ejecutados en un solo núcleo (o CPU), sin poder aprovechar el paralelismo que ofrecerían múltiples procesadores.

18. Considere un sistema multiprocesador y un programa multithreaded utilizando el modelo de threads muchos-a-muchos. Sea la cantidad de threads a nivel usuario mayor que la cantidad de procesadores en el sistema. Discuta sobre el rendimiento en las siguientes situaciones:

a) La cantidad de kernel threads alocados al programa es menor que la cantidad de procesadores.

b) La cantidad de kernel threads alocados al programa es igual a la cantidad de procesadores.

c) La cantidad de kernel threads alocados al programa es superior a la cantidad de procesadores pero menor que la cantidad de threads a nivel de usuario

19. Un multiprocesador con ocho procesadores tiene 20 unidades de cinta magnéticas. Hay un gran número de jobs (tareas) enviados al sistema donde cada uno requiere un máximo de cuatro unidades de cinta para completar su ejecución. Asuma que cada trabajo se pone en marcha con solo tres unidades de cinta durante un largo período antes de requerir la cuarta unidad de cinta por un corto período de tiempo hacia el final de su ejecución. Considere que nunca finaliza el envío de tareas de este tipo.

a) Supongamos que el planificador del sistema operativo no iniciará un trabajo a menos que tenga las cuatro unidades de cinta disponibles. Cuando un trabajo se inicia, cuatro unidades se asignan de forma inmediata y se no se liberan hasta que finalice. ¿Cuál es el número máximo de trabajos que pueden estar ejecutándose a la vez? ¿Cuales son la cantidad máxima y mínima de cintas que están ociosas con esta política?

El número máximo de trabajos que pueden estar ejecutándose a la vez es de 5, ya que si tenemos 20 unidades y cada job que haya requerirá 4, podemos hacer la cuenta $20 \div 4 = 5$. En base a esto, entonces podemos decir que la cantidad máximas de cintas ociosas con esta política es de un total de 5.

b) Sugerir una política alternativa para mejorar la utilización de unidades de cinta y al mismo tiempo evitar deadlock. ¿Cuál es el número máximo de tareas que pueden estar ejecutándose a la vez? ¿Cuáles son los límites en el número de unidades de cintas ociosas?

Una alternativa sería la de que como se usan 4 cintas en un corto periodo de tiempo, se añaden directamente 6 trabajos al principio. Después, quedan dos cintas libres, las cuales se dan a medida que se vayan necesitando. Podría pasar que un proceso necesite una cinta para terminar su ejecución y que esta no esté disponible, por lo que en el instante en que un proceso finaliza, este se retira de las cintas y en vez de traer un nuevo proceso, las cintas se asignan a los procesos que estan ejecutándose y la necesitan. Recién cuando hayan 4 cintas libres y ningún proceso necesite alguna, es cuando se trae uno nuevo. (Digo 4 y no 3 para así si un proceso en ejecución necesita una cinta, esta esté disponible, pero podría ser 3). El número máximo de tareas a la vez es 6, y el límite en el número de cintas ociosas es 3.

18. Considere un sistema multiprocesador y un programa multithreaded utilizando el modelo de threads muchos-a-muchos. Sea la cantidad de threads a nivel usuario mayor que la cantidad de procesadores en el sistema. Discuta sobre el rendimiento en las siguientes situaciones:

a) La cantidad de kernel threads alocados al programa es menor que la cantidad de procesadores.

Al tener menos threads de kernel que procesadores, no se estarían aprovechando todos los procesadores, lo que significa que, en caso de poder paralelizar más el programa, no se estaría llegando a la máxima eficiencia.

b) La cantidad de kernel threads alocados al programa es igual a la cantidad de

procesadores.

En este caso, se podría tener un hilo de kernel por procesador, lo que significa que se es más eficiente que el inciso anterior, al poder estar usando todos los procesadores al mismo tiempo.

c) La cantidad de kernel threads alocados al programa es superior a la cantidad de procesadores pero menor que la cantidad de threads a nivel de usuario.

Si bien se estarían utilizando todos los procesadores (como en el inciso anterior), además de la ventaja que eso supone, si alguno de los hilos de usuario hace una llamada al sistema bloqueante, se bloquea el thread de kernel, pero al tener más, otro hilo podría tomar su lugar y seguir ejecutando, lo que quiere decir que en todo momento se podría tener un hijo ejecutando en el procesador, lo cual es muy eficiente.